

Package ‘facereaderconverter’

August 19, 2026

Title Process of FaceReader outputs and emotional episodes

Version 0.2.4

Description Tools to convert FaceReader outputs; Markup of emotional episodes.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports dplyr,

hms,
janitor,
readr,
readxl,
stringr,
tidyselect,
tools,
tibble,
Rcpp,
data.table,
tidyr

LinkingTo Rcpp

Suggests openxlsx,
testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

RoxygenNote 8.0.0

Contents

add_delta_column	2
convertFRDirectory	3
convertFRExcelFiles	4
convertFRFiles	5
convert_to_episodes	6
delta_episodes	7
loadFRfile	8
locf	9

map_paths 10

parse_time_to_frame 10

reaction_rate 11

synchrony 12

to_seconds 13

Index 14

add_delta_column	<i>Add delta column to coding_df</i>
------------------	--------------------------------------

Description

Add delta column to coding_df

Usage

```
add_delta_column(  
  coding,  
  delta = 0.1,  
  delta_window = 0.2,  
  fps = 30L,  
  cores = 0L  
)
```

Arguments

- coding Data frame with columns: id, subject, emotion, frame (or video_time), value, or an fr_coding object
- delta Threshold for both up and down
- delta_window Window size in seconds
- fps Frames per second
- cores integer Number of threads to use. Default 0 is auto.

Value

coding_df with extra column 'delta' where 1 means up and 0 means down

Examples

```
## Not run:  
coding_df = read.csv("testdata_detailed.csv")  
add_delta_column(  
  coding_df,  
  delta_window = 0.2,  
  delta = 0.1,  
  fps = 30  
)  
  
## End(Not run)
```

convertFRDirectory	<i>Convert a directory of Facereader TXT to CSV</i>
--------------------	---

Description

Reads all Txt files in a folder and sends them through convertFRFiles.

Usage

```
convertFRDirectory(
  inpath,
  outpath = inpath,
  recursive = TRUE,
  pattern = NULL,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  save_metadata = outpath,
  metadata_filename = "metadata.csv",
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  cores = 0L,
  ...
)
```

Arguments

inpath	Path to an existing .txt file.
outpath	Path to save the csvs to defaults to the inpath
recursive	Bool as to whether to look for all files in directory (TRUE) or just the root folder (FALSE)
pattern	a regex pattern of files to test, if NULL then will look for all txt files
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names
save_metadata	save the metadata as a csv in the outpath, set to NULL to not save
metadata_filename	filename of the metadata csv
fail_codes	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
duplicate_timecodes_as_error	throws an error if there are duplicate timecodes, if FALSE then throws warning
cores	integer Number of threads to use. Default 0 is auto.
...	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRDirectory(
  inpath="directory_of_txt_files",
  outpath="directory_to_save_csvs_to",
  values_as_numeric = TRUE
)

## End(Not run)
```

convertFRExcelFiles *Load FaceReader Excel exports*

Description

Reads a FaceReader .xlsx file, detects the header row automatically, and returns the parsed data frame. The function follows the same post-processing conventions as convertFRFiles() where applicable.

Usage

```
convertFRExcelFiles(
  inpath,
  return_data = TRUE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...
)
```

Arguments

inpath	Path to an existing .xlsx file.
return_data	Return the parsed data instead of metadata.
values_as_numeric	Convert Video Time to hms and detailed values to numeric.
clean_names	Apply janitor::clean_names() to the data.
fail_codes	Add a fail_code column for detailed exports.
duplicate_timecodes_as_error	Throw an error if duplicate timecodes are found.
sheet	Excel sheet to read. Defaults to the first sheet.
...	Additional arguments passed to janitor::clean_names().

Value

Invisibly returns the parsed data when return_data = TRUE; otherwise returns metadata about the import.

Examples

```
## Not run:
convertFRExcelFiles(
  inpath = "FaceReaderOutput.xlsx",
  return_data = TRUE,
  values_as_numeric = TRUE
)

## End(Not run)
```

convertFRFiles

*Convert Facereader TXT to CSV***Description**

Reads a .txt file, detects the FaceReader header row automatically, guesses the delimiter from the header line, writes a .csv with the same basename in the same directory, and returns the path to the created CSV (invisibly).

Usage

```
convertFRFiles(
  inpath,
  filepath = paste0(tools::file_path_sans_ext(inpath), ".csv"),
  return_data = FALSE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  ...
)
```

Arguments

inpath	Path to an existing .txt file.
filepath	Path to save the csv to defaults to the inpath
return_data	Bool to return the data from the txt rather than the metadata
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names
fail_codes	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
duplicate_timecodes_as_error	throws an error if there are duplicate timecodes, if FALSE then throws warning
...	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRFiles(
  inpath="FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

convert_to_episodes	<i>Convert coding_df to episodes</i>
---------------------	--------------------------------------

Description

Convert coding_df to episodes

Usage

```
convert_to_episodes(
  coding_df,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = 30L,
  cores = 0L
)
```

Arguments

coding_df	a dataframe or otherwise from a FaceReader output. id and subject should be present.
T_up	numeric Upper threshold for entering an episode. Default: 0.2.
T_down	numeric Lower threshold for exiting an episode. Default: 0.1.
delta	numeric Minimum k-step change required, computed as the current value minus the value k frames earlier. Default: 0.1.
delta_window	numeric Time window (in seconds) used to derive k, the lag for the k-step difference rule. Default: 0.1.
min_dur_sec	numeric Minimum episode duration (in seconds). Default: 0.1.
consecutive_missing	integer Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L.
fps	integer Frames per second (sampling rate of the data). Default: 30L.
cores	integer Number of threads to use. Default 0 is auto.

Details

The function uses the exported native binding `hysteresis_state` if available; otherwise it will error. It relies on **data.table** for fast grouping and joins.

Value

A list with two elements:

- episodes** data.table of detected episodes with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, `start_time`, `end_time`, and `run_id`.
- coding** Annotated data.table containing the original columns plus `id`, `subject`, `emotion`, `value`, `run_id`, `status`, and `in_state`. `status` marks episode boundaries with 1L at the start frame and 0L at the end frame; `in_state` is TRUE for frames inside detected episodes.
- fps** Frames per second (sampling rate of the data).

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
convert_to_episodes(coding_df)

## End(Not run)
```

delta_episodes	<i>Add create delta episodes</i>
----------------	----------------------------------

Description

Add create delta episodes

Usage

```
delta_episodes(coding, fps = 30L, cores = 0L)
```

Arguments

- `coding` Data frame with columns: `id`, `subject`, `emotion`, `frame` (or `video_time`), `value`, or an `fr_coding` object
- `fps` Frames per second
- `cores` integer Number of threads to use. Default 0 is auto.

Value

episodes data.table of detected episodes with columns `start_frame`, `end_frame`, `start_time`, `end_time`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, and `run_id`.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
x=add_delta_column(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)
delta_episodes(x)

## End(Not run)
```

loadFRfile	<i>Load FaceReader files into memory</i>
------------	--

Description

Reads a FaceReader export file into memory by dispatching to the existing TXT, Excel, or CSV readers based on file extension. The wrapper always returns the parsed data and never writes an output file.

Usage

```
loadFRfile(
  inpath,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...
)
```

Arguments

<code>inpath</code>	Path to an existing FaceReader export file.
<code>values_as_numeric</code>	Convert Video Time to hms and detailed values to numeric where applicable.
<code>clean_names</code>	Apply <code>janitor::clean_names()</code> to the data.
<code>fail_codes</code>	Add a <code>fail_code</code> column for detailed exports.
<code>duplicate_timecodes_as_error</code>	Throw an error if duplicate timecodes are found.
<code>sheet</code>	Excel sheet to read when importing .xlsx files.
<code>...</code>	Additional arguments passed to the underlying importer.

Value

Invisibly returns the parsed data frame.

Examples

```
## Not run:
loadFRfile(
  inpath = "FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

locf

Apply LOCF and Extract Episodes

Description

Applies Last Observation Carried Forward (LOCF) logic to the coding data.table (from convert_to_episodes), imputing missing run_id values within blocks of non-missing value, grouped by id, subject, and emotion. Returns both the imputed coding table and a summary of contiguous episodes.

Usage

```
locf(coding, fps = 30, consecutive_missing = NULL)
```

Arguments

coding	A datatable, dataframe or fr_coding object as returned by convert_to_episodes, containing columns: id, subject, emotion, video_time, value, and run_id.
fps	Frames per second (sampling rate of the data). Default: 30L or inherited from an fr_coding object
consecutive_missing	Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L or inherited from an fr_coding object

Value

A list with three elements:

episodes data.table of contiguous episodes after LOCF, with columns start_frame, end_frame, n_frames, duration_s, id, subject, emotion, and run_id.

coding Annotated data.table with LOCF-imputed run_id.

metadata Metadata from fr_coding object.

map_paths	<i>Map files from an input root to an output root, preserve structure, create dirs.</i>
-----------	---

Description

Map files from an input root to an output root, preserve structure, create dirs.

Usage

```
map_paths(input_dir, output_dir, files)
```

Arguments

input_dir	Character scalar. The source root directory.
output_dir	Character scalar. The destination root directory.
files	Character vector. File paths under input_dir to be remapped.

Value

Character vector of output file paths (same length as files).

parse_time_to_frame	<i>Parse video time to frame index</i>
---------------------	--

Description

Convert a time string into a frame index given frames-per-second (fps). Supports "HH:MM:SS", "MM:SS", or plain seconds and is vectorised over video_time.

Usage

```
parse_time_to_frame(video_time, fps)
```

Arguments

video_time	Character or numeric. Time strings in "HH:MM:SS", "MM:SS", or numeric seconds.
fps	Numeric. Frames per second.

Value

Integer vector. Frame indices computed as round(seconds * fps).

Examples

```
parse_time_to_frame("00:01:23.5", fps = 30)
parse_time_to_frame("1:23.5", fps = 30)
parse_time_to_frame("83.5", fps = 30)
```

reaction_rate	<i>Calculate reaction rate from delta episodes</i>
---------------	--

Description

Reaction rate is the proportion of eligible episodes that contain at least one delta reaction after an initial exclusion window.

Usage

```
reaction_rate(
  coded_data,
  episode_limit = 3,
  episode_limit_frames = NULL,
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = 30L,
  subject_names = NULL,
  exclude_emotions = "neutral"
)
```

Arguments

<code>coded_data</code>	Output from <code>add_delta_column()</code> or a compatible data frame.
<code>episode_limit</code>	Maximum episode length to include in the denominator.
<code>episode_limit_frames</code>	Optional maximum episode length in frames. If supplied, this takes precedence over <code>episode_limit</code> .
<code>exclude_start</code>	Minimum reaction onset to count. Reactions that begin earlier than this are excluded from the numerator.
<code>exclude_start_frames</code>	Optional minimum reaction onset in frames. If supplied, this takes precedence over <code>exclude_start</code> .
<code>fps</code>	Frames per second used to convert between seconds and frames.
<code>subject_names</code>	Character vector of subject labels to compare. If <code>NULL</code> , all unique subjects in <code>coded_data</code> are used. If supplied, data are filtered to those subject levels before reaction rate is calculated.
<code>exclude_emotions</code>	Character vector of emotions to exclude. Default is "neutral".

Value

A `data.table` with columns `id`, `subject`, `emotion`, `n_episodes`, `n_reactions`, and `reaction_rate`.

Examples

```
library(data.table)

coded_data <- data.table(
  id = rep(1L, 8),
```

```

subject = rep(c("teen", "parent"), each = 4),
emotion = "happy",
frame = rep(1:4, 2),
delta = c(1L, 1L, 0L, 1L, 0L, 1L, 1L, 0L)
)
reaction_rate(coded_data, fps = 30)

```

synchrony

*Calculate synchrony from converted episodes***Description**

Calculate synchrony from converted episodes

Usage

```

synchrony(
  coded_data,
  subject = "subject",
  id = "id",
  missing_threshold = 0,
  exclude_emotions = "neutral"
)

```

Arguments

<code>coded_data</code>	Output from <code>convert_to_episodes()</code> .
<code>subject</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the subject. Default is "subject".
<code>id</code>	Character scalar giving the column in <code>coded_data\$coding</code> and <code>coded_data\$episodes</code> that identifies the case or dyad. Default is "id".
<code>missing_threshold</code>	Numeric scalar in $[0, 1]$. Denominator and numerator episodes are dropped when the comparison subject is missing for more than this proportion of frames within the episode. Default is 0.
<code>exclude_emotions</code>	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, and `synchrony`.

Examples

```

library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),

```

```

    value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
    in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
    run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
  )
  episodes <- data.table(
    id = 1L,
    subject = c("teen", "parent"),
    emotion = "happy",
    run_id = c(1L, 2L),
    start_frame = c(2L, 3L),
    end_frame = c(2L, 3L)
  )
  coded_data <- structure(
    list(coding = coding, episodes = episodes),
    class = c("fr_coding", "list")
  )
  synchrony(coded_data, subject = "subject", id = "id", missing_threshold = 0)

```

to_seconds

Convert FaceReader time strings to seconds

Description

Parses time values formatted as hh:mm:ss or hh:mm:ss.mmm and returns the equivalent number of seconds.

Usage

```
to_seconds(x)
```

Arguments

x Character vector of time strings.

Value

A numeric vector of seconds.

Examples

```
to_seconds(c("00:00:10", "00:00:10.500"))
```

Index

`add_delta_column`, [2](#)

`convert_to_episodes`, [6](#)

`convertFRDirectory`, [3](#)

`convertFRExcelFiles`, [4](#)

`convertFRFiles`, [5](#)

`delta_episodes`, [7](#)

`loadFRfile`, [8](#)

`locf`, [9](#)

`map_paths`, [10](#)

`parse_time_to_frame`, [10](#)

`reaction_rate`, [11](#)

`synchrony`, [12](#)

`to_seconds`, [13](#)